

Platform Engineeringはなぜ スケールしないのか

スコープ設計と責任境界の話

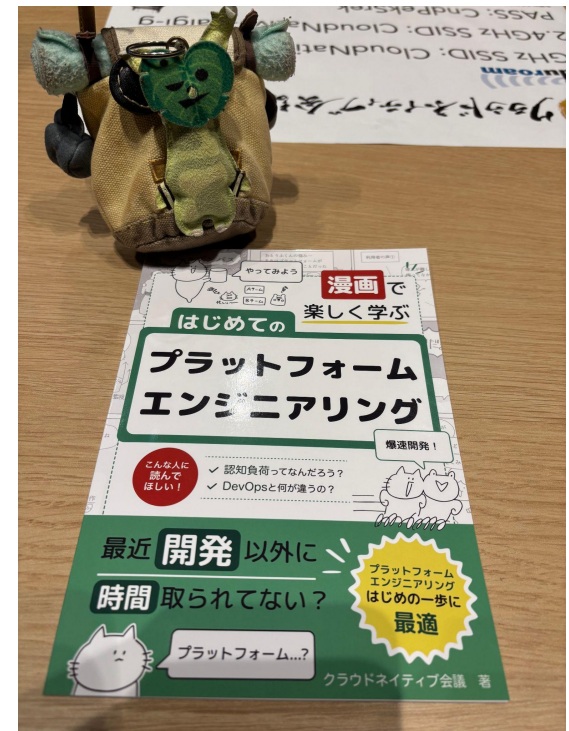
木嶋幸子 @Red Hat

今日配布のプラットフォームエンジニアリング本

3章の内容だけ時間いっぱいお話しします！

(持っていない方へ: プロダクトマネジメント部分です)

わかりやすい内容でした、著者の皆様ありがとうございます





木嶋 幸子 (Sachiko KIJIMA) @Red Hat

- ▶ エンタープライズアーキテクチャ
- ▶ アーキテクチャモダナイゼーション
- ▶ プラットフォームエンジニアリング

2020/5から現職 (Red Hatコンサルティング部門)



Platform Engineering のプロジェクトで我々がやっていること

各種プラクティスの定着支援

- ▶ システム思考/デザイン思考/顧客中心主義
- ▶ CI/CD (テスト・デプロイ自動化)
- ▶ XP
- ▶ スクラム/LEAN
- ▶ エンタープライズアジャイル
- ▶ エンタープライズアーキテクチャ

Platform Engineering的な成果とは

特性		暫定的である	戦略的である	スケーラブルである	最適化している
投資	プラットフォームの機能に対して、人員と資金がどのように割り当てられているか	ボランティアまたは一時的	専任チーム	プロダクト	活発なエコシステム
採用	ユーザーはなぜ、どのようにして内部プラットフォームやその機能を発見し、利用しているか	一貫していない	外部からの動機づけ	内発的な動機づけ	プロアクティブな参加
インターフェース	ユーザーがどのようにしてプラットフォームの機能と対話し、利用しているか	機能毎に独自の手順	標準ツール	セルフサービスのソリューション	統合されたサービス
戦略	プラットフォームとその機能が、どのように計画され、優先順位がつけられ、開発され、維持されているか	リクエストに応じて	中央集権的な追跡	中央集権的な支援	マネージドサービス
計測	どのようなプロセスでフィードバックと学びを収集し、取り入れているか？	アドホック	一貫性を持った収集	洞察の獲得	定量的かつ定性的

Platform Engineeringはたいてい組織変革

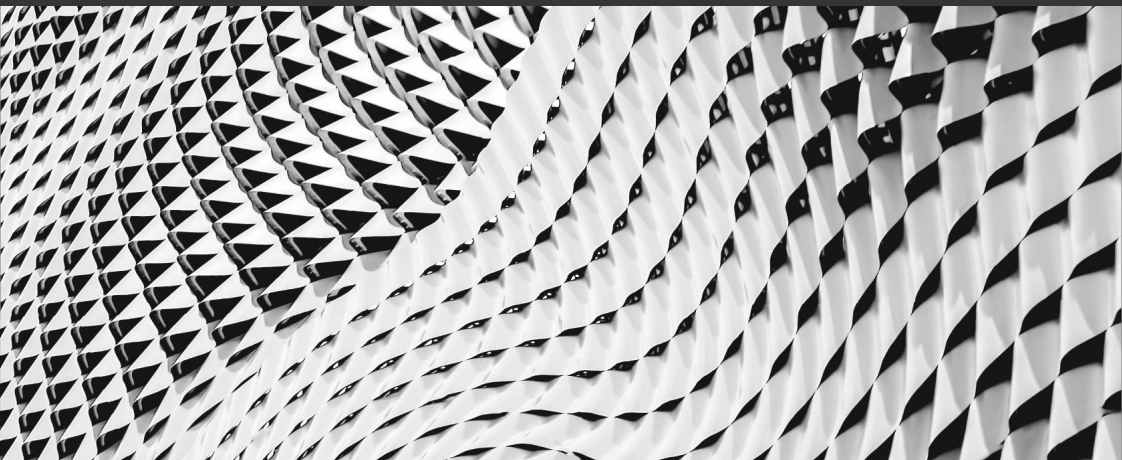
... なので同じ教科書が参考になる



Kotterをベースにした失敗要因分析

No	Kotterのステップ	失敗パターン例
1	危機意識を高める	なぜやるかが明確ではない、あるいは定義されていない 共感をうまない「危機」が設定されている
2	変革推進チームを作る	チーム組成の失敗 チームは組成されたが、横断的チームにならず、周囲が協力的にならない
3	ビジョンと戦略を生み出す	実行メンバーにわかりにくい、あるいは共感しにくいビジョンが決まってしまう 効果の実感しにくい戦略が立案されてしまう、あるいは戦略そのものがない
4	変革のためのビジョンを周知徹底する	メンバー自身が忘れる 周囲が目的を誤解し期待値を押し付ける
5	従業員の自発を促す	メンバーが「自分ごと」化できない、やらされ感につながる 考えても行動に移せない
6	短期的成果を実現する	周囲の期待に振り回されてWinを確立できない Winを周囲が認識してくれない
7	成果を生かして、さらなる変革を推進する	Winが単発で終わる
8	新しい方法を企業文化に定着させる	メンバーにナレッジが蓄積されない RHがリードしすぎてRHがいないと前に動かない

1. 危機意識を高める



1. 危機意識を高める: ねらいと失敗パターン

ねらい

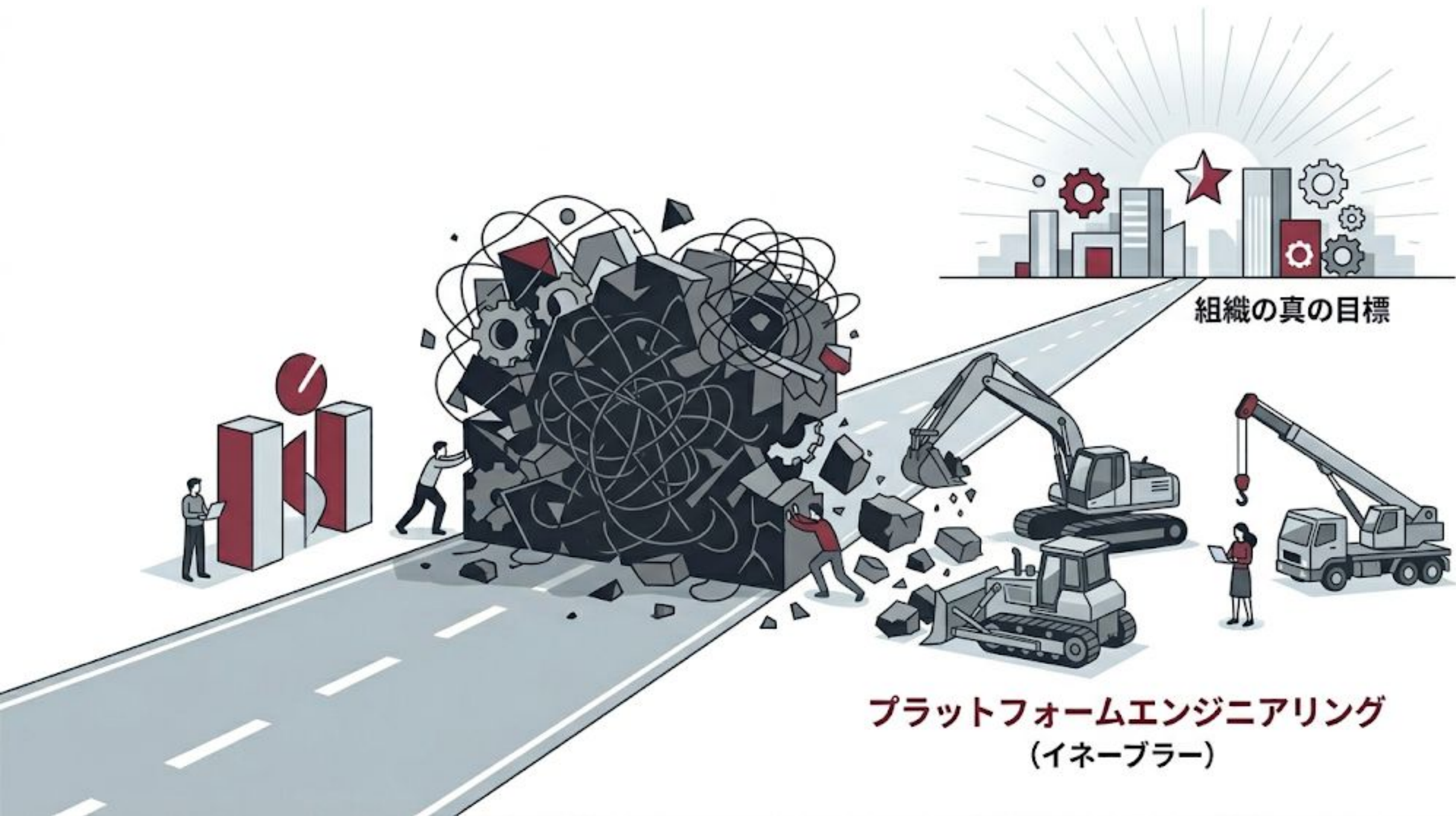
- ▶ みんながこの活動を自分ごと化できる
- ▶ みんなが同じ方向を向いて活動できる

あるある失敗パターン

乗り越えたい危機やなぜこの活動をしたいのかが不明瞭なため、

- ▶ 偉い人から予算や人がもらえない
- ▶ ステークホルダーの関心を得られない
- ▶ 実行メンバーがやるべきということが統一できない

結果、スコープがばらけたり、チームが空中分解したりする

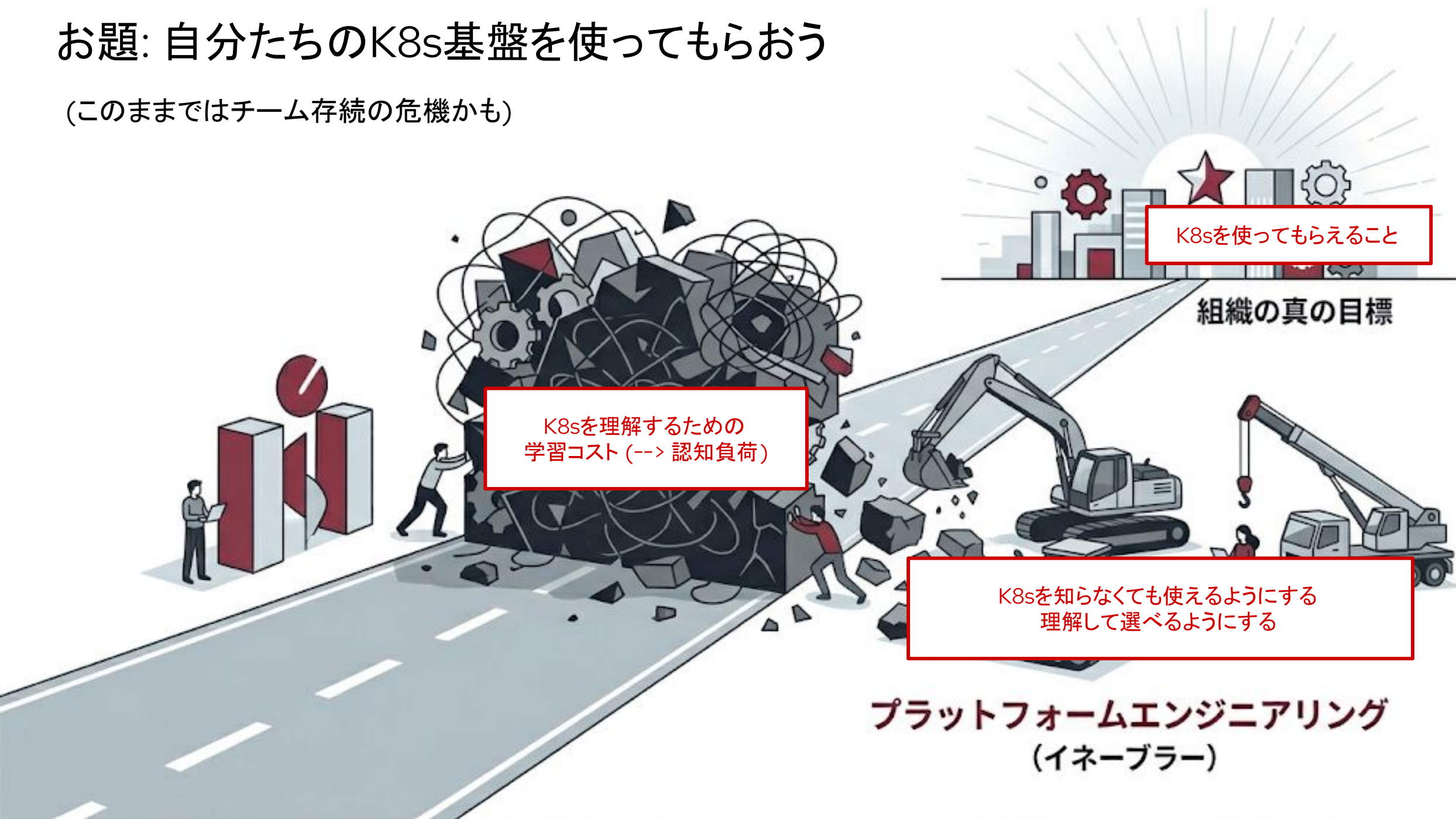


組織の真の目標

プラットフォームエンジニアリング
(イネーブラー)

お題: 自分たちのK8s基盤を使ってもらおう

(このままではチーム存続の危機かも)



Step 1: 穴埋め問題

認知負荷のせいで、ユーザーは [自分たちのシステムでK8sを利用可能か判断できない] ことに困っている。

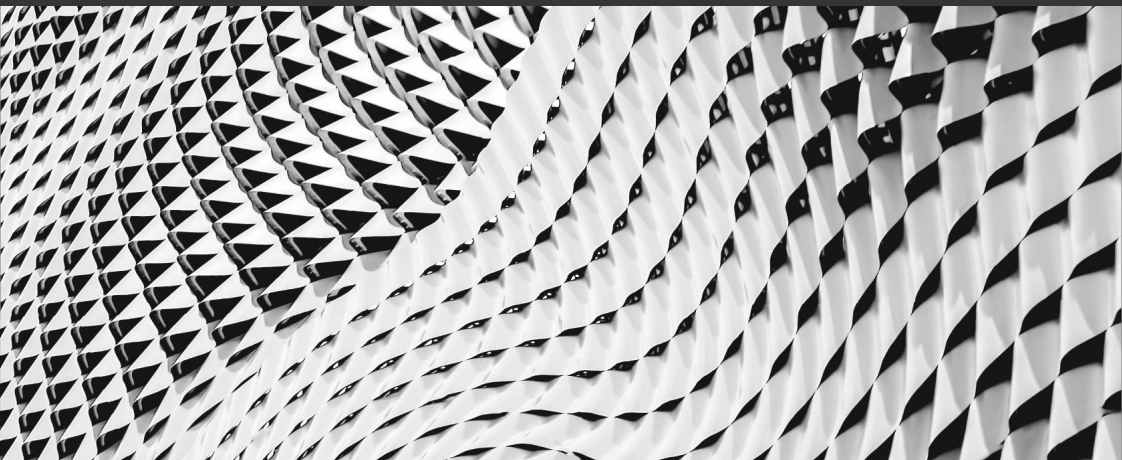
それは、プラットフォームチームにとって [作った基盤が採用されない] という問題を生んでいる。

<この行のみ任意> さらにこれは、経営者にとって [基盤投資の効果が見えず、継続投資を判断しにくい] という問題に見えている。

これを放置すると、[基盤が停止され、K8s基盤がリタイアさせられてしまう] 事態が発生する。

この負の連鎖を解消するために、[K8sを知らなくとも使えるようにする] プラットフォームサービスを用意することで [K8sを使ってもらえる] ようにする必要がある。

3. ビジョンと戦略を生み出す



3. ビジョンと戦略を生み出す: ねらいと失敗パターン

ねらい

- ▶ 変化が起きなければいけない領域を特定できる
- ▶ 誰の、どんな困りごとを解消したらその変化が起きるかを特定できる
- ▶ 変化を起こすためにどんな方法で介入すべきかを特定できる
- ▶ どんな方法は責任範囲外とすべきかが特定できる

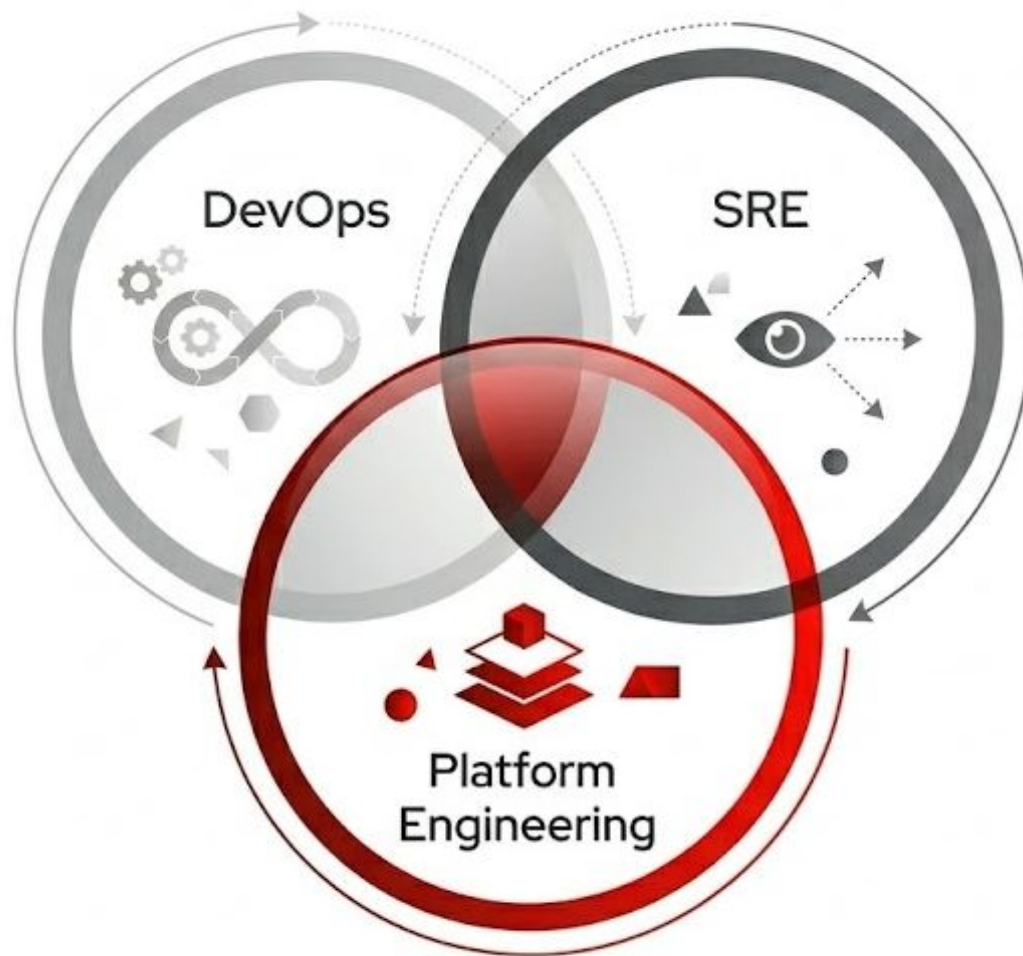
あるある失敗パターン

戦略がないことで、

- ▶ スコープが定義できない
- ▶ ご用聞き化
- ▶ 多様な期待値を押し付けられて何をすべきかわからない

結果が出にくくて嫌になったり、
チームが空中分解したりする

Platform Engineeringはスコープがブレやすい



実際にみた活動例

- ▶ **プロダクト寄り**: ポータル開発・申請書解説ドキュメント作成
- ▶ **プロジェクト支援**: CI/CD構築・トラブルシューティング・アーキテクチャ設計
- ▶ **標準化寄り**: 標準ドキュメント作成、標準技術検討

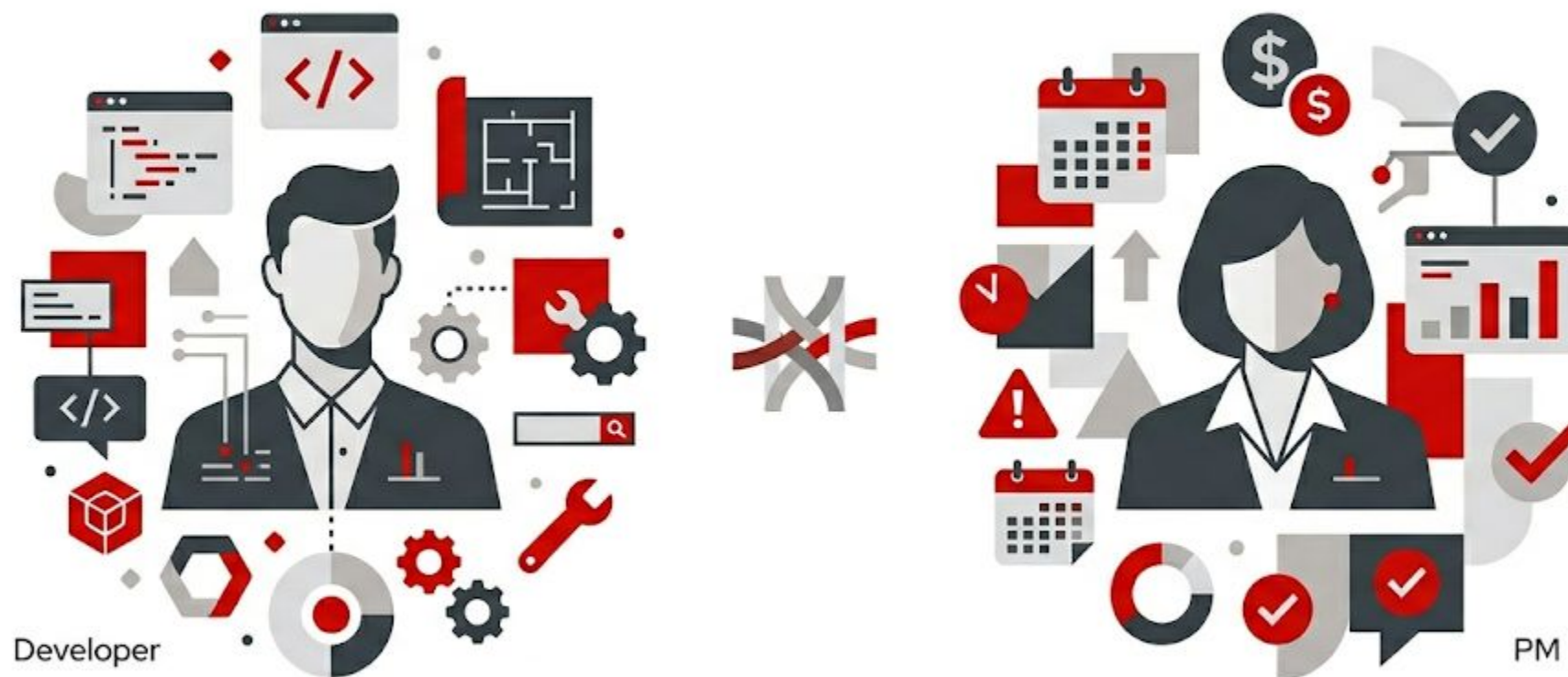
戦略 = Platform as a Product の確立

- ▶ **ペルソナ**: 顧客は誰か
- ▶ **価値**: その人は何に困っていて、どんな価値を必要としているか
- ▶ **スコープ**: 何をやり、何をやらないのか
- ▶ **提供資産**: どんな再利用可能なプラットフォームサービスを作るのか

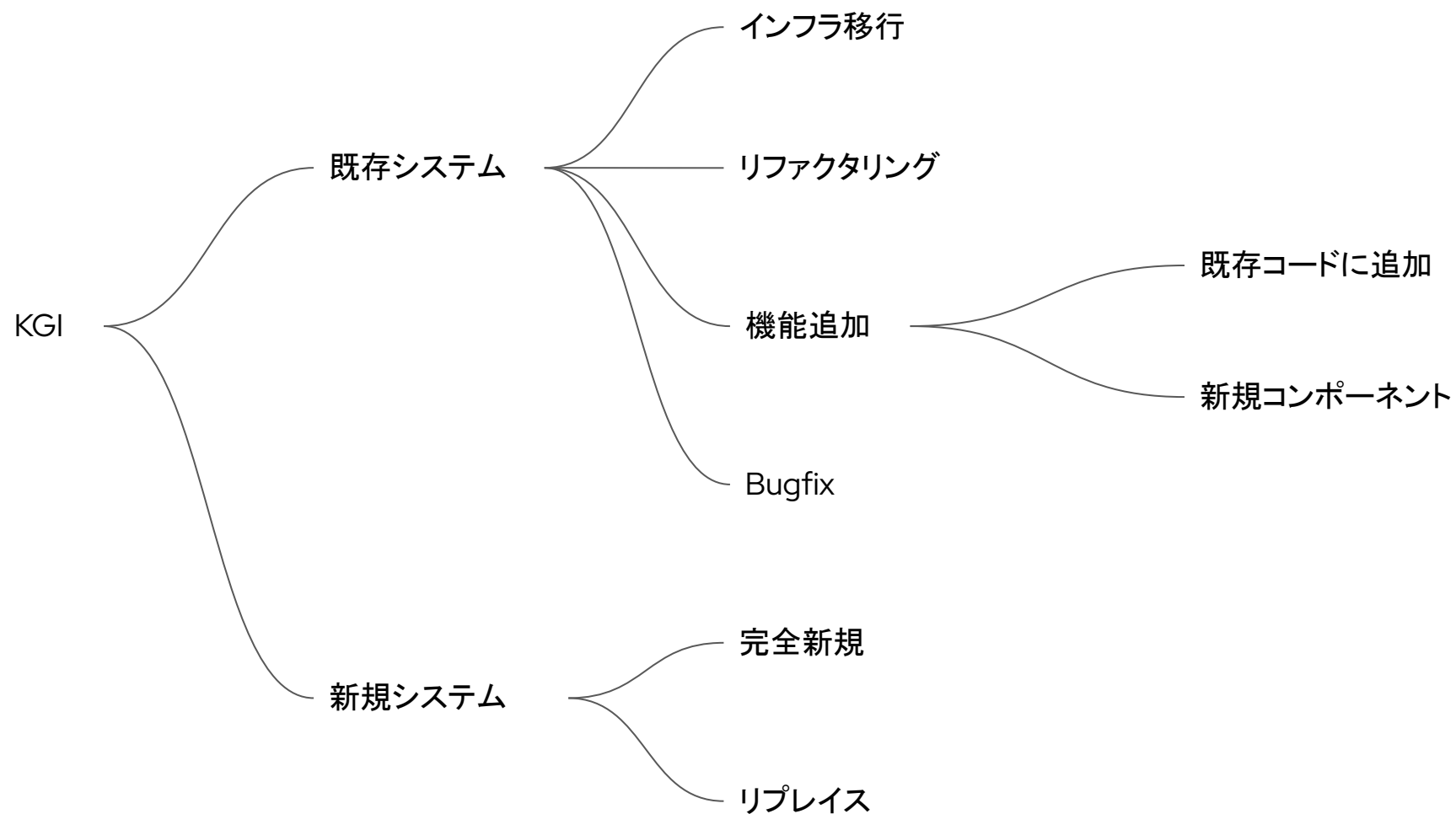
→ 「みんなが何度も繰り返し使うもの」を厳選する

→ 「下手な鉄砲数打ちゃ当たる」を避ける

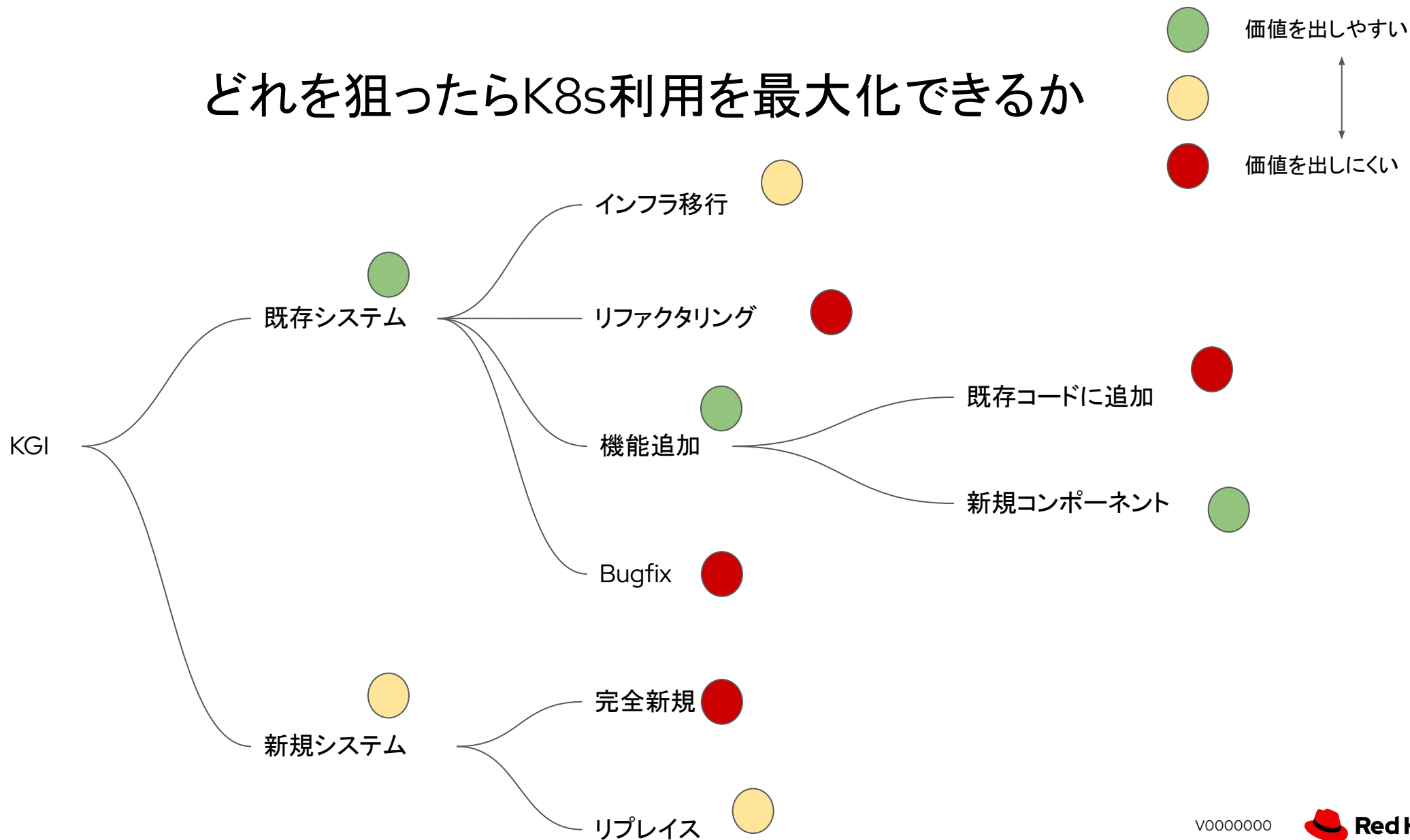
対象ペルソナは本当に開発者か



パターン分析



どれを狙ったらK8s利用を最大化できるか



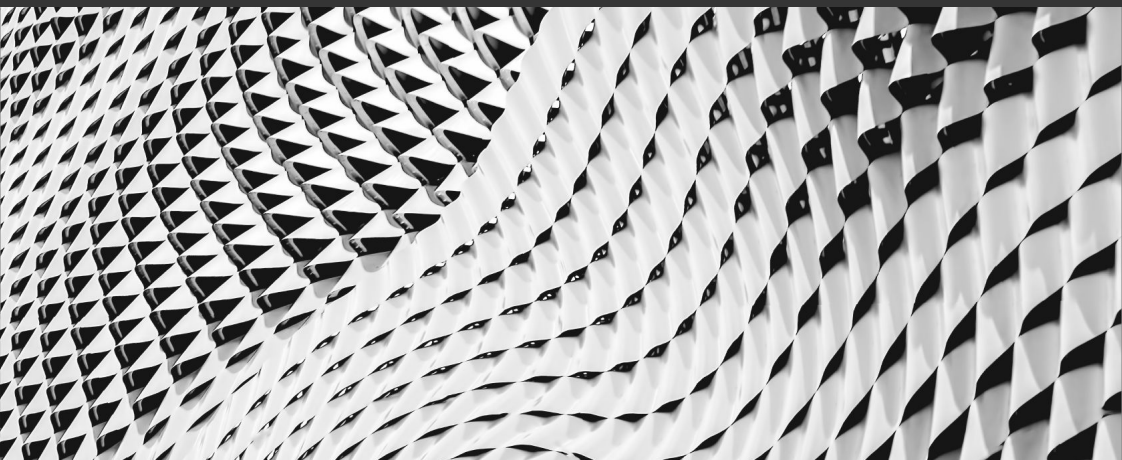
Step 3: 穴埋め問題

[既存システムに新規機能を追加するために新しいコンポーネントを開発する開発者に対して必要な情報を提供するプロジェクトマネージャー]にとって、[開発者ポータル (IDP)] は、[やることや調整をすることがわかりやすくまとまった] リストを提供します。

このリストは、[プロジェクトマネージャーが標準のプラットフォームサービスを適切に利用し、会社のルールに則したアプリケーションを、大きな苦勞なく設計からデプロイメント完了まで到達できるフロー] を提供します。

今までのポータルの情報と異なり、今回の機能は [プロジェクトマネージャーの視点でわかりやすくまとまっており、このページをワンストップとして使えるものとなっています。]

6. 短期的成果を実現する (Quick Win)



6. 短期的成果を実現する: ねらいと失敗パターン

ねらい

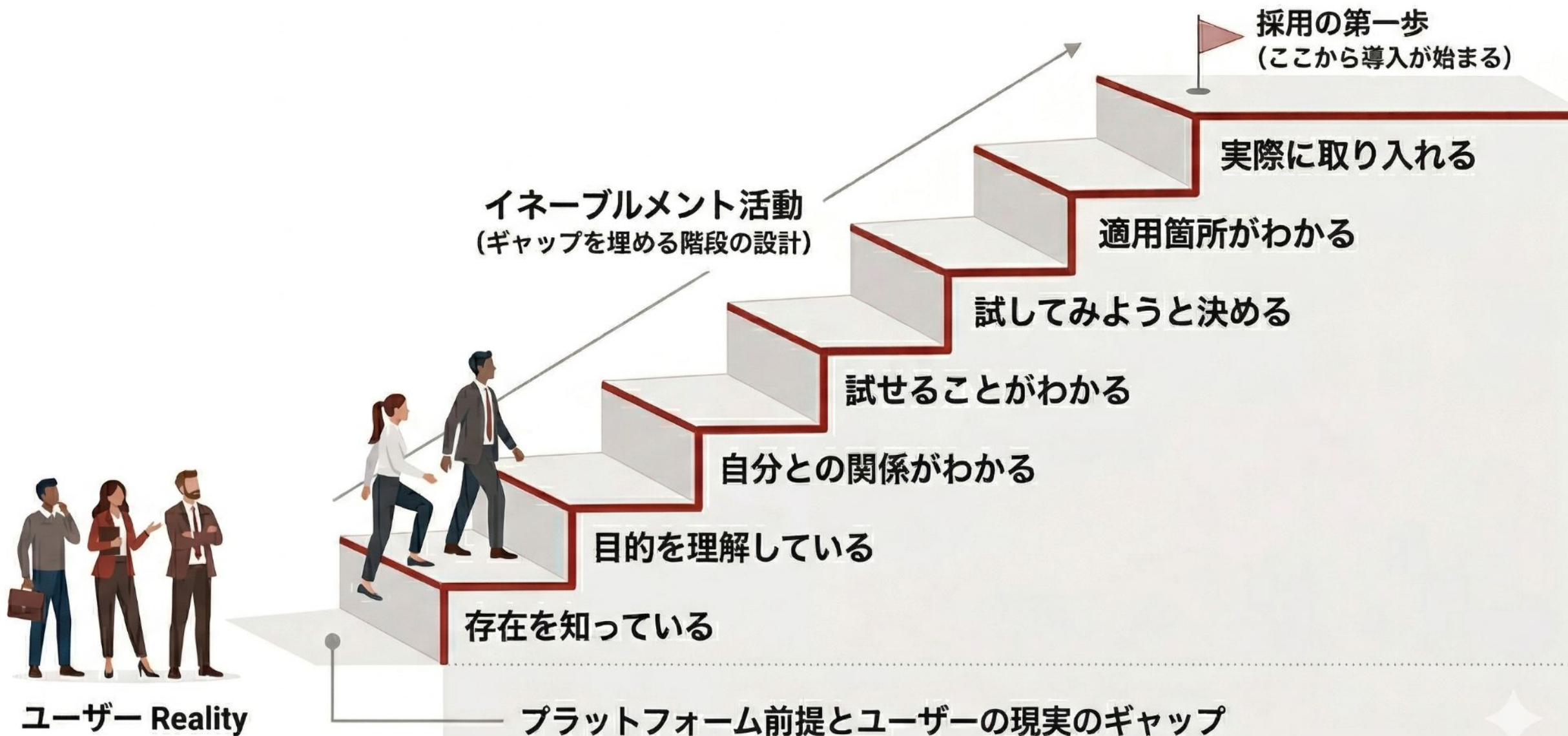
- ▶ **外向き**: 周囲に「期待できる」と思ってもらう。採用を意識してもらう
- ▶ **内向き**: チームが「続ける意味がある」と実感できる

あるある失敗パターン

せっかく成功を収めても周知しないことで、

- ▶ Winが周囲に認知されず、採用や支援に繋がらない
- ▶ チームにとっても手応えがないので、「頑張っても意味が見えない」状態になる

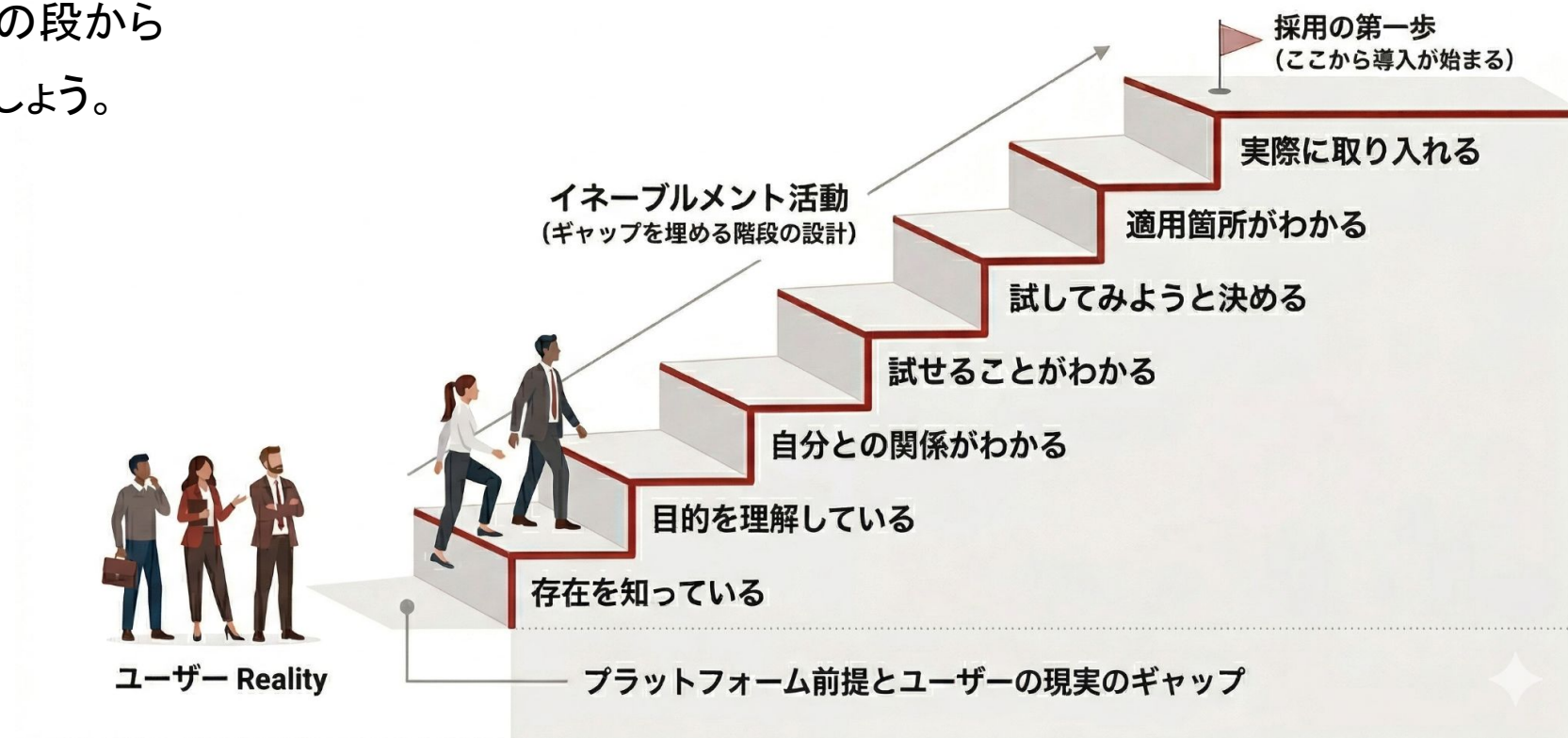
活動の意味を見失って
チームが空中分解したりする



Step 6: 穴埋め問題

自分たちの階段を作しましょう！

広報やイネーブルメントごとに、どの段から
どの段に連れていくのか確認しましょう。



相手の視点につながる説明にすること！

仮想化基盤の特徴

仮想サーバー

思う通りに作れるが、建てるのに時間がかかり、管理コストも高い



特徴:

- ▶ フルカスタマイズ可能
- ▶ 構築・起動が遅い
- ▶ 管理・維持コストが高い

コンテナエンジン

ある程度型が決まっているがすぐ使える。ただし生活の基盤は自分もち



特徴:

- ▶ 標準化が基本
- ▶ 高速に構築・起動
- ▶ ネットワークやログ、セキュリティは自前で準備

コンテナオーケストレーション

ある程度型が決まっているがすぐ使える。生活の基盤もある程度見てくれる



特徴:

- ▶ 複数コンテナを一元管理
- ▶ 自動復旧・負荷分散等、基本的な運用の仕組みがすでにある
- ▶ ネットワークやログ、セキュリティ等の仕組みを標準化して提供しやすい

V0000000



ケストレーション

CONFIDENTIAL designator

アプリケーション

開発者が注力すべき唯一の領域

↓ オフロード ↓

実行 / 運用機能

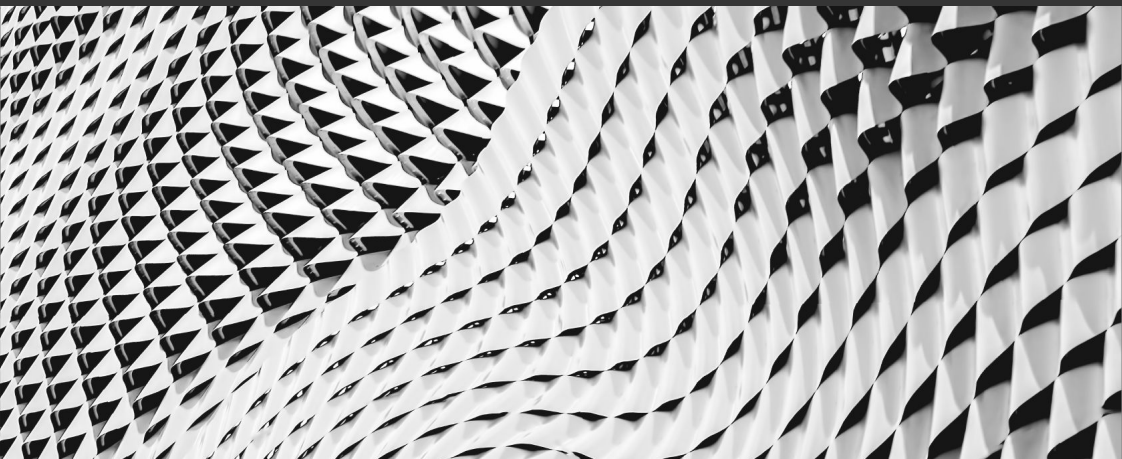
(、監視、セキュリティ、デプロイ)

基盤

(コンテナオーケストレーション / OpenShift)

- ▶ **開発者の解放:** 運用機能を基盤に任せることで、アプリ開発に専念。
- ▶ **標準化の恩恵:** 誰が作っても同じ運用レベルを担保可能。
- ▶ **認知負荷の削減:** 覚えるべき「インフラの作法」を最小化。

まとめ



- 目的の明確化
- 自分ごと化
- 共感



危機意識 / Why / Goal

- 定性の概観
- 成分の視化
- 会備

- Change Management
- 泥臭い
- Collaboration



組織変革 / 泥臭い改善活動



Transformation

Platform as a Product



顧客定義 / ペルソナ

- 開発者
- PM
- ステークホルダー
- ニーズ



境界線
スコープ定義



提供資産

- Reference Architecture
- Design Catalog
- Templates
- Guides
- Patterns



Enablement / Adoption / 階段



短期的成果



継続的な見直し

長期的な道のり
Transformation の達成
1年後・3年後の変化
Mature Ecosystem
Foundations
• 持続可能
• スケール

おまけ



<https://github.com/skijima404/pe-failure-lab>

Platform Engineeringの失敗パターンをモデリングしました。

今自分がやっていることが失敗するとしたら、どんな失敗か、回避するために何をしたら良いかを確認できます。

リポジトリごと生成AIに渡し、Step 1の穴埋め問題や、今やっていることなどを渡すと、Quick Scanができます。



実際にうまくいっている事例が何をどんなふうにしたかを詳しくお話しくさいます。

もしよろしければお立ち寄りください。

Thank you

Red Hat is the world's leading provider of enterprise open source software solutions. Award-winning support, training, and consulting services make Red Hat a trusted adviser to the Fortune 500.



linkedin.com/company/red-hat



youtube.com/user/RedHatVideos



facebook.com/redhatinc



twitter.com/RedHat